

Tài liệu thiết kế · v1.0

21/08/2026

DebtFlow — Thiết kế Full-stack

Hệ thống quản lý mua hàng & công nợ nhà cung cấp đa cơ sở
Backend + Frontend + Database — chuẩn triển khai production

NestJS

Prisma

PostgreSQL 16

React + Vite + TS

TanStack Query

Zustand

JWT + RBAC

Docker + Nginx

Nâng cấp từ bản demo frontend (HTML/CSS/JS + localStorage) lên nền tảng full-stack đầy đủ backend, cơ sở dữ liệu quan hệ và bảo mật production.

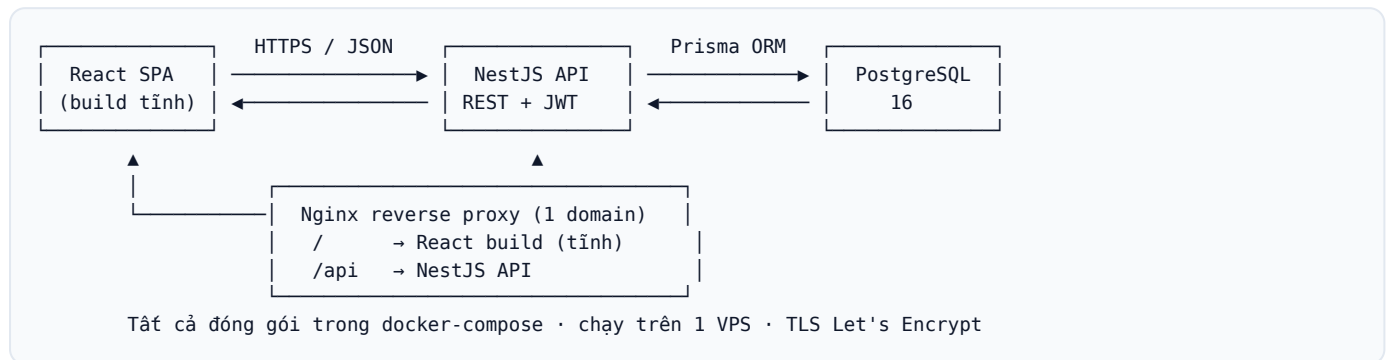
Mục lục

- 1. Tổng quan & Kiến trúc hệ thống
- 2. Công nghệ sử dụng (Tech stack)
- 3. Phân tích hướng đối tượng (Domain Model / OOP)
- 4. Cơ sở dữ liệu (PostgreSQL schema)
- 5. Thiết kế API (NestJS modules & endpoints)
- 6. Luồng nghiệp vụ chính (Business flows)
- 7. Danh sách Page & chức năng (Frontend)
- 8. Bảo mật & phân quyền
- 9. Triển khai (Docker / Deploy)
- 10. Lộ trình thực hiện

1. Tổng quan & Kiến trúc

DebtFlow là hệ thống quản lý **mua hàng & công nợ nhà cung cấp** cho doanh nghiệp nhiều **cơ sở/chi nhánh**. Hệ thống bao trùm toàn bộ vòng đời: đặt hàng → duyệt → nhập kho → sinh công nợ → thanh toán, kèm quản lý **kho Nhập–Xuất–Tồn, thống kê chi phí, phân quyền nhân viên** và **nhật ký kiểm toán** (audit log).

Sơ đồ kiến trúc tổng thể



Nguyên tắc thiết kế

Kiến trúc phân tầng

Controller → Service (business) → Repository (Prisma) → DB.
Mỗi tầng một trách nhiệm, dễ test và thay thế.

Domain thuần tách biệt

Logic tính công nợ & tồn kho port từ `logic.js` hiện có sang module TypeScript **không phụ thuộc framework** — tái dùng & unit-test dễ.

Stateless API

Xác thực bằng JWT, không lưu session server → scale ngang được về sau.

Toàn vẹn dữ liệu tài chính

Thao tác sinh công nợ chạy trong **transaction**; huỷ thanh toán là **soft-void** (không xoá cứng); mọi thay đổi đều ghi audit.

So với bản demo hiện tại

Hạng mục	Demo hiện tại	Bản full-stack
Lưu trữ	localStorage trình duyệt	PostgreSQL (bền, backup, transaction)
Xác thực	So khớp mật khẩu dạng chữ thường	JWT + bcrypt + refresh token
Frontend	1 file <code>app.js</code> ~180KB	React component hoá, TypeScript
Nghiệp vụ	Chạy client-side	API server + kiểm soát quyền phía server
Đa người dùng	Cục bộ 1 máy	Nhiều người, dữ liệu tập trung

2. Công nghệ sử dụng

Tầng	Công nghệ	Vai trò
Frontend	React 18 + Vite + TypeScript	SPA, build tĩnh, type-safe
	React Router	Điều hướng trang
	TanStack Query	Server-state, cache, đồng bộ dữ liệu
	Zustand	State cục bộ (auth, UI)
	CSS hiện tại (giữ bảng màu DebtFlow)	Giữ nguyên UI/UX & nhận diện — sidebar xanh, 6 màu, không gradient
Backend	NestJS (TypeScript)	REST API, kiến trúc module/OOP
	class-validator / class-transformer	Validate & transform DTO
	Passport-JWT	Xác thực token
ORM / DB	Prisma + PostgreSQL 16	Truy vấn type-safe, migration, transaction
Bảo mật	JWT access + refresh, bcrypt	Đăng nhập, băm mật khẩu
	RBAC + Permission Guard	Phân quyền theo vai trò & module/action
Kiểm thử	Jest/Vitest, Supertest, Playwright	Unit domain, e2e API, smoke FE
Hạ tầng	Docker, docker-compose, Nginx	Đóng gói & reverse proxy + TLS
CI/CD	GitHub Actions	Lint → test → build image

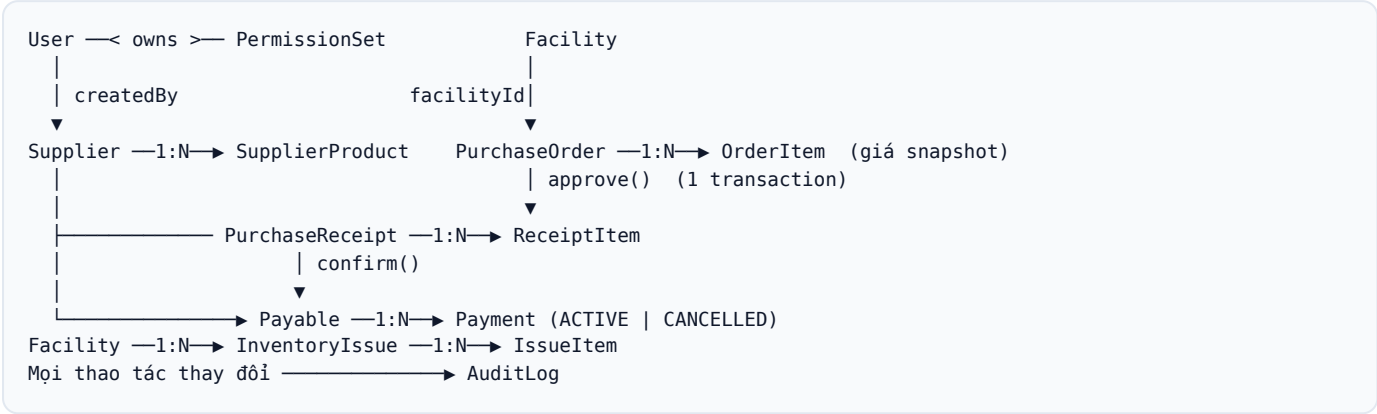
Cấu trúc thư mục dự kiến (monorepo)

```
debtflow/
├── apps/api/           # NestJS backend
│   ├── src/
│   │   ├── auth/ users/ facilities/ suppliers/
│   │   ├── orders/ receipts/ payables/ payments/
│   │   ├── inventory/ reports/ audit/ settings/
│   │   ├── domain/    # logic thuần: calculators, ledger
│   │   └── prisma/    # PrismaService
│   └── prisma/schema.prisma
├── apps/web/          # React frontend
│   └── src/{pages, components, api, store, styles}
├── packages/shared/   # types & enums dùng chung FE+BE
├── docker-compose.yml
└── nginx/ .github/workflows/
```

3. Phân tích hướng đối tượng (Domain Model)

Mô hình chia thành các **Aggregate** (nhóm thực thể có ràng buộc chung, thao tác qua một gốc), các **Entity**, **Value Object** và **Domain Service** (logic thuần không I/O).

Sơ đồ quan hệ thực thể



Aggregate roots & Entity

- **User** — vai trò **ADMIN** / **STAFF** ; Staff gắn *PermissionSet*.
- **Facility** — cơ sở/chi nhánh.
- **Supplier** → sở hữu nhiều **SupplierProduct** (danh mục + giá).
- **PurchaseOrder** → **OrderItem[]**; giá **snapshot** lúc tạo.
- **PurchaseReceipt** → **ReceiptItem[]**.
- **Payable** — 1:1 với Receipt.
- **Payment** — soft-void được.
- **InventoryIssue** → **IssueItem[]**.
- **AuditLog** — bất biến (append-only).

Domain Service (thuần, port từ logic.js)

- **InvoiceCalculator**
invoiceBalance(), invoiceStatus()
- **PurchaseTotalsCalculator**
tạm tính, giảm giá, VAT, tổng
- **InventoryLedger**
inventoryReport() NXT, canIssue() chặn xuất vượt tồn
- **PeriodCalculator**
periodBounds, previousPeriodBounds, percentChange

Giữ nguyên công thức đã được kiểm thử ở bản demo → độ tin cậy nghiệp vụ cao.

Máy trạng thái (State machines)

Thực thể	Trạng thái & chuyển tiếp
PurchaseOrder	PENDING → APPROVED (sinh Receipt+Payable) REJECTED (kèm lý do) CANCELLED (staff huỷ khi chưa duyệt)
PurchaseReceipt	DRAFT → CONFIRMED (sinh Payable)
Payable (tính runtime)	UNPAID / PARTIAL / PAID / OVERDUE — suy ra từ Payments & dueDate
Payment / InventoryIssue	ACTIVE → CANCELLED (soft-void, hoàn số dư/tồn)

Bất biến nghiệp vụ (Invariants)

- Không cho xuất kho vượt tồn khả dụng (**canIssue**).
- Giá đơn đã chốt không đổi khi sửa danh mục NCC về sau (snapshot).
- Duyệt đơn sinh Receipt + Payable trong **một transaction** — hoặc thành công trọn vẹn, hoặc rollback.
- Dữ liệu tài chính không xoá cứng — chỉ soft-void kèm audit.

4. Cơ sở dữ liệu (PostgreSQL)

Tiền: `NUMERIC(18,2)` · Số lượng: `NUMERIC(14,3)` (hỗ trợ 12.5 kg) · Khoá ngoại ràng buộc toàn vẹn · **Payable status không lưu cứng** — tính runtime để luôn đúng theo payments/ngày.

Bảng	Cột chính	Quan hệ
users	id, name, email🔑, password_hash, role, status, last_login_at	1:N orders/receipts...
staff_permissions	user_id, module, action, allowed (hoặc JSONB trên users)	N:1 users
facilities	id, code🔑, name, address, status	1:N orders/issues
suppliers	id, code🔑, name, phone, email, tax_code, contact_person, address, note, status	1:N products/receipts
supplier_products	id, supplier_id, name, unit, price, status, note	N:1 suppliers
purchase_orders	id, order_code🔑, supplier_id, facility_id, status, note, expected_date, created_by, reviewed_by, reviewed_at, reject_reason, result_receipt_id, result_payable_id, created_at	1:N order_items
order_items	id, order_id, product_id, name, unit, unit_price, quantity (snapshot)	N:1 orders
purchase_receipts	id, receipt_code🔑, supplier_id, facility_id, supplier_invoice_code, receipt_date, due_date, status, discount_amount, tax_amount, note, created_by, confirmed_by, created_at	1:N receipt_items · 1:1 payable
receipt_items	id, receipt_id, item_name, unit, quantity, unit_price, note	N:1 receipts
payables	id, invoice_code, supplier_id, purchase_receipt_id, invoice_date, due_date, total_amount, description, note, created_by, created_at	1:N payments
payments	id, direction, payable_invoice_id, amount, payment_date, payment_method, transaction_code, note, status, created_by, created_at, cancelled_by, cancelled_at	N:1 payables
inventory_issues	id, issue_code🔑, facility_id, issue_date, note, status, created_by, created_at, cancelled_by, cancelled_at	1:N issue_items
issue_items	id, issue_id, item_name, unit, quantity	N:1 issues
audit_logs	id, time, user_id, action, entity_type, entity_id, detail	append-only
settings	warning_days, critical_warning_days, currency, timezone (singleton)	—

Trích lược Prisma schema

```
model PurchaseOrder {
  id          String    @id @default(cuid())
  orderCode   String    @unique
  supplier    Supplier  @relation(fields:[supplierId], references:[id])
  supplierId  String
  facilityId  String
  status      OrderStatus @default(PENDING)
  items       OrderItem[]
  createdBy   String
  reviewedBy  String?
  rejectReason String?
  resultReceiptId String?
  resultPayableId String?
  createdAt   DateTime  @default(now())
}

enum OrderStatus { PENDING APPROVED REJECTED CANCELLED }
```

5. Thiết kế API (NestJS)

Tiền tố `/api/v1` · bảo vệ bằng `JwtGuard` + `PermissionGuard` · trả JSON chuẩn, lỗi có mã & message rõ ràng.

Module	Endpoint tiêu biểu
Auth	POST <code>/auth/login</code> · POST <code>/auth/refresh</code> · GET <code>/auth/me</code>
Users	GET/POST/PATCH <code>/users</code> · PUT <code>/users/:id/permissions</code>
Facilities	GET/POST/PATCH <code>/facilities</code>
Suppliers	GET/POST/PATCH <code>/suppliers</code> · GET <code>/suppliers/:id</code> (tổng công nợ)
SupplierProducts	GET/POST/PATCH <code>/suppliers/:id/products</code>
Orders	GET/POST <code>/orders</code> · GET <code>/orders/:id</code> · POST <code>/orders/:id/approve</code> · <code>/reject</code> · <code>/cancel</code>
Receipts	GET/POST <code>/receipts</code> (DRAFT) · POST <code>/receipts/:id/confirm</code>
Payables	GET <code>/payables</code> (balance/status tính sẵn) · GET <code>/payables/:id</code>
Payments	POST <code>/payments</code> · POST <code>/payments/:id/void</code>
Inventory	GET/POST <code>/inventory/issues</code> · POST <code>/issues/:id/cancel</code> · GET <code>/inventory/report</code> · POST <code>/inventory/check</code>
Reports	GET <code>/reports/dashboard</code> · GET <code>/reports/stats</code> · GET <code>/reports/compare</code>
Audit / Settings	GET <code>/audit-logs</code> · GET/PUT <code>/settings</code>

Ví dụ luồng gọi API — Duyệt đơn

```
// Admin bấm "Duyệt" đơn ord-001
POST /api/v1/orders/ord-001/approve
→ Service chạy trong 1 transaction:
  1. đọc order (khóa dòng), kiểm tra status == PENDING
  2. tạo PurchaseReceipt(CONFIRMED) từ order.items
  3. tạo Payable(totalAmount) gán receipt
  4. cập nhật order: status=APPROVED, result_receipt_id, result_payable_id
  5. ghi AuditLog(action=APPROVE_ORDER)
← 200 { order, receipt, payable } (hoặc rollback toàn bộ nếu lỗi)
```

6. Luồng nghiệp vụ chính

① Đặt hàng → Duyệt (luồng lỗi)

Staff tạo đơn (chọn **cơ sở** + **NCC**, nhập số lượng — giá **snapshot** tự điền) → đơn ở **PENDING**. Staff có thể **hủy** khi chưa duyệt. **Admin Duyệt** ⇒ hệ thống (1 transaction) sinh **Phiếu nhập CONFIRMED** + **Công nợ phải trả** + ghi audit. Hoặc **Từ chối** kèm lý do. Sidebar admin hiện số đơn chờ duyệt.

② Nhập hàng trực tiếp

Tạo **Phiếu nhập** nhiều dòng (tên hàng, ĐVT, SL, đơn giá, giảm giá, VAT) ở **DRAFT** → **Xác nhận** ⇒ sinh **Công nợ** tương ứng + audit.

③ Công nợ → Thanh toán

Màn công nợ hiển thị **số dư** & **trạng thái tính runtime**. Tạo **Thanh toán** (một phần / đủ, kèm phương thức & mã GD) → trạng thái tự chuyển UNPAID → PARTIAL → PAID. **Void** thanh toán ⇒ hoàn số dư + audit.

④ Kho Nhập – Xuất – Tồn

Phiếu nhập CONFIRMED = **nhập**; Phiếu xuất ACTIVE = **xuất**. Báo cáo: **Tồn cuối = Tồn đầu + Nhập – Xuất** theo cơ sở & khoảng ngày; xem **thẻ kho** từng mặt hàng; lập phiếu xuất **chặn vượt tồn**.

⑤ Cảnh báo công nợ

Dựa **warningDays** / **criticalWarningDays** → badge "sắp đến hạn" & "quá hạn" trên Dashboard và sidebar; tô màu theo mức độ.

⑥ Kiểm toán (Audit)

Mọi hành động thay đổi (đăng nhập, duyệt, tạo/void thanh toán, sửa danh mục...) ghi **AuditLog** — bảng append-only phục vụ tra soát.

7. Danh sách Page & chức năng (Frontend)

Giữ đầy đủ 18 màn của bản demo, UI/UX & bảng màu DebtFlow không đổi.

#	Page	Chức năng chính
1	Đăng nhập	Email + mật khẩu → JWT; điều hướng theo vai trò
2	Dashboard	4 KPI + so sánh kỳ trước, chọn 1/3/6/12 tháng, so sánh cơ sở, cảnh báo dùng vượt tháng, biểu đồ SVG
3	Nhà cung cấp (list)	Tìm kiếm, tổng phát sinh / còn nợ / quá hạn mỗi NCC, thêm/sửa
4	Chi tiết NCC	4 tab: Mặt hàng · Phiếu nhập · Công nợ · Thanh toán
5	Mặt hàng	Danh mục + lịch sử nhập theo giá
6	Đặt hàng	List (lọc cơ sở), Tạo đơn, Chi tiết & duyệt/từ chối
7	Phiếu nhập	List, Tạo nhiều dòng, Chi tiết
8	Công nợ	List phải trả, Chi tiết + tạo thanh toán
9	Lịch sử thanh toán	Danh sách + huỷ (void) giao dịch
10	Thống kê	Sản lượng/chi phí theo cơ sở & kỳ, xuất CSV
11	So sánh kỳ	Đối chiếu chi phí nhập 2 kỳ
12	Kho NXT	Báo cáo Nhập–Xuất–Tồn, thẻ kho, lập phiếu xuất
13	Audit Log	Nhật ký hành động toàn hệ thống
14	Người dùng	Quản lý user & phân quyền theo module/action
15	Cài đặt	Cơ sở (thêm/sửa/ẩn), cấu hình cảnh báo nợ, đơn vị tiền

Xuất CSV/Excel chuẩn UTF-8 có BOM được giữ nguyên trên các bảng báo cáo.

8. Bảo mật & phân quyền

- **Xác thực:** JWT access (~15') + refresh (~7 ngày, cookie httpOnly), mật khẩu băm bcrypt.
- **Phân quyền:** RBAC — **ADMIN** toàn quyền; **STAFF** theo *PermissionSet* (module → action: view/edit/pay...) do admin cấp, kiểm tra tại *PermissionGuard* phía server.
- **An toàn dữ liệu:** validate DTO đầu vào, rate-limit đăng nhập, CORS chặt, ẩn thông tin nhạy cảm khỏi response.
- **Toàn vẹn tài chính:** transaction cho thao tác sinh công nợ; soft-void thay xoá cứng; audit đầy đủ.

9. Triển khai (Docker / Deploy)

```
# docker-compose.yml (rút gọn)
services:
  postgres: image: postgres:16 · volume bên · cron pg_dump backup
  api:      NestJS · tự chạy prisma migrate deploy khi khởi động
  nginx:    serve React build (/) + proxy /api → api · TLS Let's Encrypt
```

- Một domain duy nhất: `/` → frontend, `/api` → backend.
- **Seed script:** tạo tài khoản admin đầu tiên + dữ liệu mẫu tùy chọn.
- Backup định kỳ `pg_dump`; biến môi trường qua `.env` (secret, DB URL, JWT key).

10. Lộ trình thực hiện

Giai đoạn	Nội dung
1	Prisma schema + migration + seed · dựng khung NestJS/React
2	Auth + RBAC + Permission Guard
3	Master data: NCC · cơ sở · sản phẩm · người dùng
4	Đặt hàng → duyệt (transaction sinh Receipt + Payable)
5	Phiếu nhập · Công nợ · Thanh toán (+ void)
6	Kho Nhập–Xuất–Tồn (+ chặn xuất vượt tồn)
7	Reports · Dashboard · Thống kê · So sánh kỳ
8	Audit · Settings · hoàn thiện FE từng module
9	Docker · CI/CD · deploy VPS · backup · tài liệu vận hành

— Hết tài liệu thiết kế · DebtFlow Full-stack v1.0 —